

Functions

A function takes values in and gives one back. `give` is the word for handing a value back.

```
use std.io

fn area(width: Num, height: Num) -> Num {
    give width * height
}

fn main() {
    io.say("{area(3.0, 4.0)}")
}
```

Parameter types are always written down. The result type goes after `->` ; leave it off when the function gives nothing back.

Short functions

When the whole body is one expression, use `=>` :

```
use std.io

fn double(n: Int) -> Int => n * 2
fn shout(words: Text) -> Text => words.upper() + "!"

fn main() {
    io.say("{double(21)}")
    io.say(shout("quietly"))
}
```

Functions as values

A function can be stored, passed and returned.

```
use std.io

fn apply_twice(value: Int, change: fn(Int) -> Int) -> Int {
    give change(change(value))
}

fn main() {
    let add_three = fn(n: Int) => n + 3
    io.say("{apply_twice(10, add_three)}")
}
```

A function written inline like that is a lambda. It can see the values around it:

```
use std.io

fn main() {
    let tax = 0.2
    let with_tax = fn(price: Num) => price * (1.0 + tax)
    io.say("{with_tax(100.0)}")
}
```

Lambdas capture by value, so `tax` is fixed at the moment the lambda is made. Nothing changes under you later.

Pipelines

`|>` sends the value on the left into the function on the right. It turns a nest of calls inside out.

```
use std.io

fn main() {
    let total = [1, 2, 3, 4, 5] |> sum
    io.say("{total}")
    io.say("{ " hello " |> trim |> upper}")
}
```

Exercise

Write `celsius_to_fahrenheit` and `fahrenheit_to_celsius`, then print a small conversion table from -10 to 40 in steps of 10.